

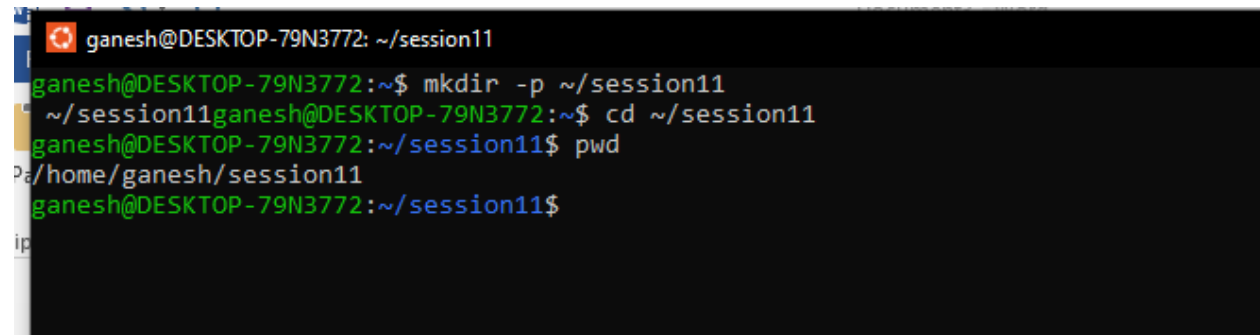
Session - 11

TASK 1 — Multithreaded Counter and Race Condition

Question

Develop a multithreaded counter application using POSIX threads (`pthread_create`, `pthread_join`). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Create folder

A terminal window with a dark background and light green text. The prompt is 'ganesh@DESKTOP-79N3772: ~/session11'. The user enters 'mkdir -p ~/session11', then 'cd ~/session11', and finally 'pwd'. The output of 'pwd' is '/home/ganesh/session11'.

```
ganesh@DESKTOP-79N3772: ~/session11
ganesh@DESKTOP-79N3772:~$ mkdir -p ~/session11
~/session11ganesh@DESKTOP-79N3772:~$ cd ~/session11
ganesh@DESKTOP-79N3772:~/session11$ pwd
/home/ganesh/session11
ganesh@DESKTOP-79N3772:~/session11$
```

Create a c program

```

#include <stdio.h>
#include <pthread.h>

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter value: %d\n", counter);
}

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U U
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E R

```

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter value: %d\n", counter);

    return 0;
}

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy

```

ganesh@DESKTOP-79N3772:~/session11$ nano race.c
ganesh@DESKTOP-79N3772:~/session11$ gcc race.c -o race -pthread
ganesh@DESKTOP-79N3772:~/session11$ ./race
Final counter value: 134153
ganesh@DESKTOP-79N3772:~/session11$ ./race
/race
./race
./race
./race
Final counter value: 151364
ganesh@DESKTOP-79N3772:~/session11$ ./race
Final counter value: 200000
ganesh@DESKTOP-79N3772:~/session11$ ./race
Final counter value: 147314
ganesh@DESKTOP-79N3772:~/session11$ ./race
Final counter value: 143928
ganesh@DESKTOP-79N3772:~/session11$ ./race
Final counter value: 146285
ganesh@DESKTOP-79N3772:~/session11$

```

output after running ./race several times

Observation

Multiple threads accessed the shared counter simultaneously, resulting in a race condition and inconsistent output.

TASK 2 — Mutex Synchronization

Question

Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the output with the race-condition version. Analyze the impact of synchronization.

Create a mutex program

```
GNU nano 8.7.1 mutex.c *
#include <stdio.h>
#include <pthread.h>

int counter = 0;

pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final counter value: %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^Y Exit      ^R Read File  ^\ Replace    ^U Paste      ^_ Justify
```

Compile

```

ganesh@DESKTOP-79N3772:~/session11$ nano mutex.c
ganesh@DESKTOP-79N3772:~/session11$ gcc mutex.c -o mutex -pthread
ganesh@DESKTOP-79N3772:~/session11$ ./mutex
Final counter value: 200000
ganesh@DESKTOP-79N3772:~/session11$

```

Comparing both programs

Race condition and mutex synchronization

```

ganesh@DESKTOP-79N3772:~/session11$ ./race
Final counter value: 100000
ganesh@DESKTOP-79N3772:~/session11$ ./mutex
Final counter value: 200000
ganesh@DESKTOP-79N3772:~/session11$

```

```

Final counter value: 200000
ganesh@DESKTOP-79N3772:~/session11$ echo "Race Condition:"
ce

echo "Mutex Synchronization:"
./mutexRace Condition:
ganesh@DESKTOP-79N3772:~/session11$ ./race
Final counter value: 141465
ganesh@DESKTOP-79N3772:~/session11$
ganesh@DESKTOP-79N3772:~/session11$ echo "Mutex Synchronization:"
Mutex Synchronization:
ganesh@DESKTOP-79N3772:~/session11$ ./mutex
Final counter value: 200000
ganesh@DESKTOP-79N3772:~/session11$

```

`pthread_create()` → creates a new thread.

`pthread_join()` → waits for a thread to finish.

Race condition → occurs when multiple threads access shared data concurrently without proper synchronization.

Mutex → provides mutual exclusion so only one thread can access the critical section at a time.

`pthread_mutex_lock()` → locks the mutex.

`pthread_mutex_unlock()` → unlocks the mutex.

Observation: Mutex synchronization prevented race conditions and produced the correct counter value consistently.